Hexo Labs

SIA: Self Improving AI with Harness & Weight Updates

Prannay Hebbar*, Yogendra Manawat*, Samuel Verboomen, Alesia Ivanova,
Selvam Palanimalai, Kunal Bhatia, Vignesh Baskaran

Updating both the scaffold and the weights in a single self-improving loop

Humans are the bottleneck

- Today's AI progress is rate-limited by **people**: models are post-trained by researchers; agents are scaffolded, prompted, and debugged by engineers.
- The long-horizon goal — an AI that can figure out how to **improve itself** — remains open.

One concrete step

Given only a **task specification** and a **verifier**, SIA improves *both* its scaffold *and* its model weights — with no further human intervention.

Two silos of self-improving AI — and the gap

Silo 1 — Harness / scaffold

A meta-agent rewrites the scaffold (prompts, tool dispatch).

Model weights **fixed**.

Darwin Gödel Machine, Meta-Harness, Hyperagents.

Silo 2 — Test-time training

A hand-written RL pipeline updates the model's weights on task feedback.

Harness **fixed**.

TTRL, Discover-TTT.

The gap: the two silos operate in *isolation*. SIA turns **both** knobs in one loop.

Research questions

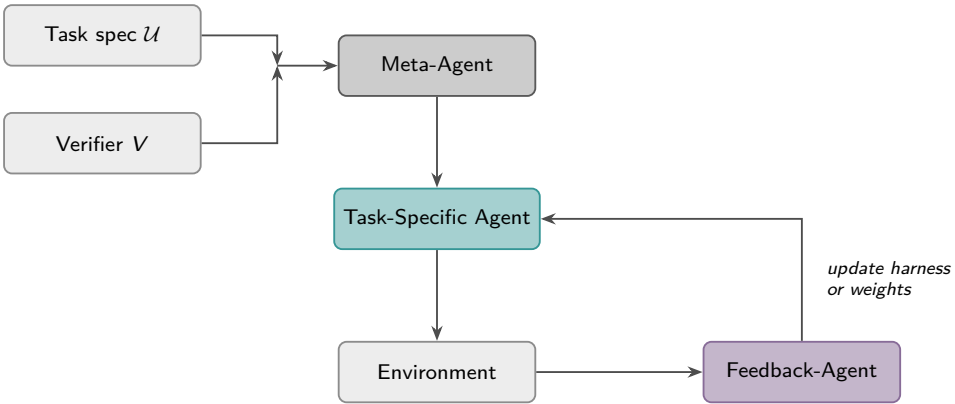
RQ1 — Overall thesis

How much does harness iteration alone help when weights are fixed? And does running *both* levers together push past that harness-only ceiling — across contrasting domains?

RQ2 — Mechanism: what does each lever change?

Do weight updates surface domain knowledge no scaffold edit reaches? Does harness iteration produce qualitatively different (external infrastructure) changes?

SIA system architecture



The loop repeats until the step budget is exhausted. Meta-Agent and Feedback-Agent use `Claude Sonnet 4.6`; the task agent uses `gpt-oss-120b` (or an RL-adapted checkpoint).

Anatomy of a task-specific agent

A **task-specific agent** takes a task instance and produces an answer. The **scaffold** (= harness) is the union of the system prompt, tool-dispatch logic, answer extraction, and any supporting infrastructure, every part of the agent that is fixed code rather than model output:

- **LLM** — weights θ (`gpt-oss-120b`)
- **System prompt** — frames the task
- **Tool-dispatch logic** — routes tool calls
- **Answer extraction** — response $\rightarrow$ prediction
- **Grader** — deterministic verifier / reward

scaffold = system prompt + dispatch + extraction + supporting infra

Two meta-agents

A **meta-agent** is an LLM call whose output is itself an agent.

Meta-Agent $\mathcal{M}$ — initialises the scaffold

$$A_1 = \mathcal{M}(\mathcal{U}, \mathcal{R})$$

from task spec $\mathcal{U}$ and reference implementations $\mathcal{R}$.

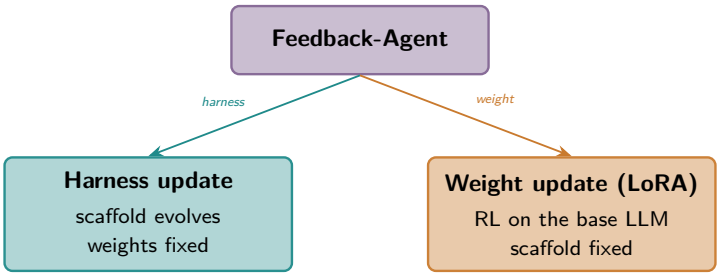
Feedback-Agent $\mathcal{F}$ — synthesises the next generation

$$A_{g+1} = \mathcal{F}(A_g, \tau_g, \mathcal{E}_g, \mathcal{U})$$

from the previous scaffold A_g, full trajectory τ_g, and metrics $\mathcal{E}_g$.

Method: two levers, one loop

After each execution, the Feedback-Agent *dynamically* selects the next action:



These two are **interleaved freely**, not locked into sequential phases. The choice is conditioned on task type and observed reward dynamics.

Prior work turns one knob; SIA turns both.

Lever 1 — Harness updates

When $\mathcal{F}$ selects a harness update, one scaffold-evolution step runs (Execution $\rightarrow$ Analysis $\rightarrow$ Improvement):

$$A_{g+1} = \mathcal{F}\big(A_g,\, \tau_g(\pi_\theta),\, \mathcal{E}_g,\, \mathcal{U}\big)$$

- Rollouts come from the current model π_θ (base or RL-adapted); **weights held fixed**, only the scaffold A_g changes.
- **Sample-task regularisation:** the Meta-Agent is conditioned on a diverse set of task specs to avoid overfitting the initial scaffold.

What harness iteration produces (RQ2a)

Externalised changes — new tools, tighter parsers, search procedures, retry policies. The model checkpoint is unchanged.

Lever 2 — Weight updates

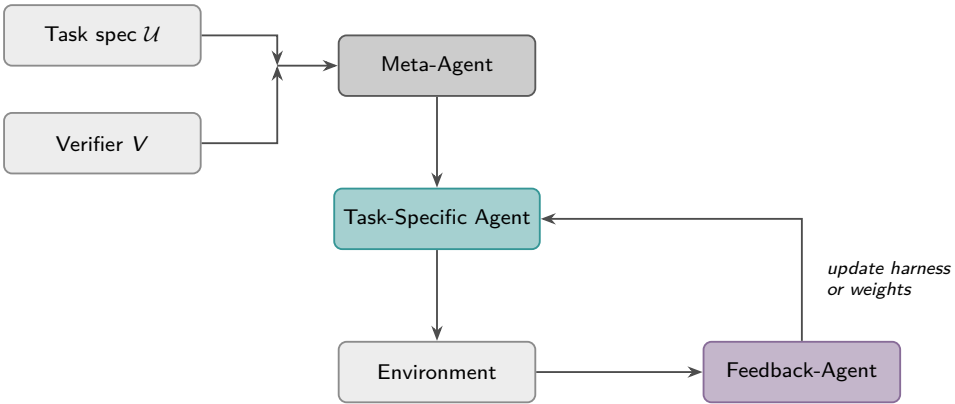
Weight updates produce *internalised* knowledge: domain-specific patterns encoded into parameters that **no scaffold edit reaches**.

- All weight updates adapt `gpt-oss-120b` via **LoRA** (rank r=32) on H100s.
- $\mathcal{F}$ does *not* run a fixed RL procedure — it selects the algorithm from trajectory observations.

Algorithm selection (general observed patterns)

PPO + GAE	dense rewards, stability is the binding constraint
GRPO	cheap rollouts, verifier fires at episode end
Entropic adv. weighting	right-skewed reward, rare high-signal solutions
REINFORCE + KL-to-base	dense reward, risk is capability regression
Best-of-N BC	reward so sparse $\mathbb{E}[r] \approx 0$ (cold-start)
DPO	verifier ranks but cannot score absolutely

SIA system architecture



The loop repeats until the step budget is exhausted. Meta-Agent and Feedback-Agent use `Claude Sonnet 4.6`; the task agent uses `gpt-oss-120b` (or an RL-adapted checkpoint).

What SIA delivers

Across three contrasting domains, combining both levers beats scaffold iteration alone — and beats prior SOTA:

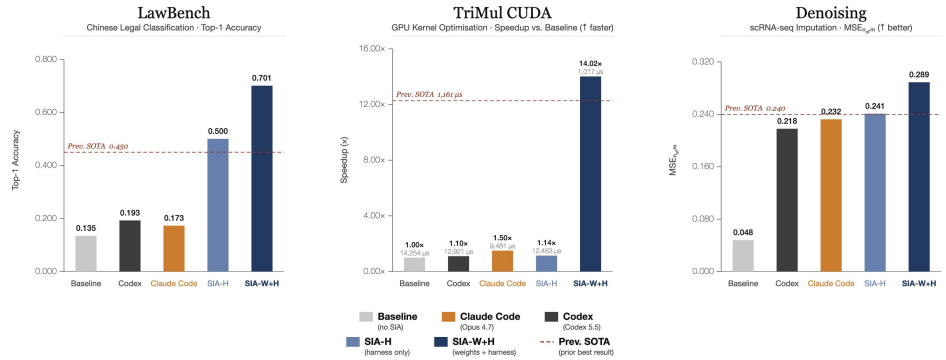
+25.1%
LawBench top-1
(70.1% vs 45.0%)

12.4% faster
GPU kernels
(1,017 vs 1,161 μs)

+20.4%
scRNA denoising
(0.289 vs 0.240)

- **Harness updates** make the model *agentic* — shaping how it searches and acts.
- **Weight updates** build *domain intuition* that no prompt or scaffold can instill.

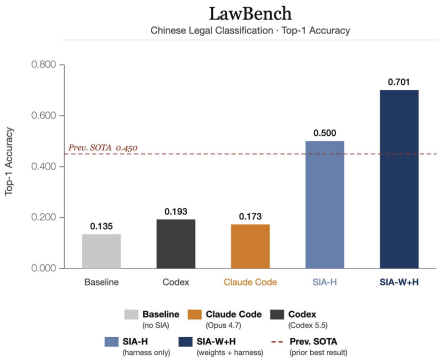
SIA across three diverse tasks



Baseline (no SIA) → SIA-H (harness only) → SIA-W+H (harness + weights). Dashed line = prior SOTA.
SIA-W+H strictly outperforms SIA-H on all three tasks.

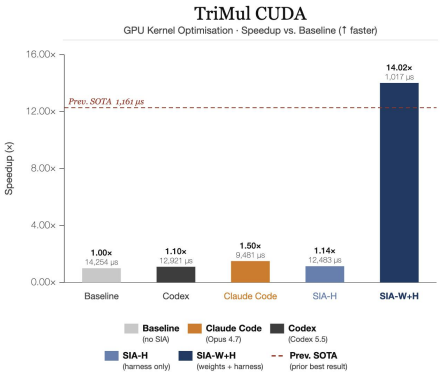
Result — LawBench (191-class Chinese charge classification)

- 191 fine-grained statutory charges; random guess $< 1\%$. 5,332 train / 913 test.
- **Harness:** TF-IDF + LinearSVC pipeline, tuned n-gram range & regulariser $C \rightarrow$ **50.0%** ($+36.5$ pp).
- **Weights (PPO+GAE):** clipped surrogate over the generated solution script $\min(r_t \hat{A}_t,\ \mathrm{clip}(r_t, 1 \pm \varepsilon)\hat{A}_t) \rightarrow$ **70.1%** ($+20.1$ pp).



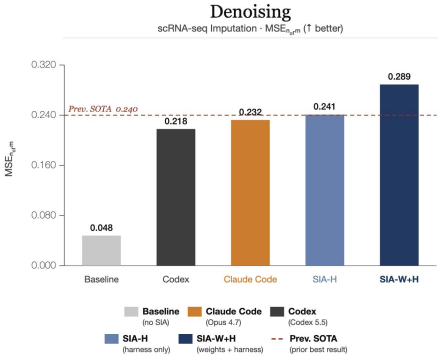
Result — AlphaEvolve TriMul (CUDA kernel optimisation)

- Triangular multiplicative update from AlphaFold2.
- **Harness:** working CUDA kernels, memory-layout hints, compile flags $\rightarrow$ 1.14$\times$ speedup.
- **Weights (entropic adv. weighting):** internalised knowledge of H100 design patterns — shared-memory tiling, fp32 register accumulation $\rightarrow$ runtime **1,017** μ**s**, **14.0**$\times$ speedup.



Result — MAGIC scRNA-seq denoising

- Tune MAGIC's coupled hyperparameters, number of neighbours k, diffusion steps t, bandwidth α on pancreas scRNA-seq data.
- **Harness:** swept the coupled hyperparameter space of MAGIC, neighbours k, diffusion steps t, bandwidth $\alpha \rightarrow$ plateau at `mse_norm` **0.241**.
- **Weights (GRPO):** first checkpoint added a structural transformation step the scaffold *never* found — a two-line post-processing step (`np.clip + np.rint`) that rounds imputed counts to non-negative integer, enforcing a biological invariant/constraint $\rightarrow$ **0.289** (+20%).



Ablation: SIA-H vs. SIA-W+H (RQ1)

Task	Initial	Prev. SOTA	SIA-H	SIA-W+H
LawBench (top-1 acc)	13.5%	45.0%	50.0%	**70.1%**
AlphaEvolve TriMul (reward)	0.105	1.292	0.120	**1.475**
Denoising (`mse_norm`)	0.048	0.240	0.241	**0.289**

- **SIA-W+H strictly outperforms SIA-H on every task** — confirming RQ1.
- **+20.1 pp** (LawBench), **91.9%** runtime reduction (TriMul), **+20%** (denoising).
- Each lever occupies a distinct change space — neither saturates the other's gains.

What each lever changes (RQ2)

Harness → *how* it searches

Externalised infrastructure:

- LawBench: answer-extraction layer + SVC re-ranker
- TriMul: compile-error parser + timing harness
- MAGIC: batched config driver + result parser
- In all three cases, the changes are software-engineering improvements: new tools, tighter output parsers, smarter retry logic.

Weights → *what* it knows

Internalised, verifier-aligned knowledge:

- LawBench: sharper disambiguation of adjacent charges
- TriMul: H100-specific kernel design patterns
- MAGIC: a biological invariant/constraint baked into the policy
- The internalised knowledge emerges from direct gradient pressure

The harness shapes **how** the agent searches; weight updates change **what** the model knows.

Limitations

Coupled co-evolutionary Goodhart

Harness search and RL weight updates both optimise against the *same* fixed verifier V:

- The harness finds scaffolds easy for the current policy to exploit.
- The weights train on data collected through a scaffold that will then change.

The joint fixed point is a **Nash equilibrium** between two optimisers blind to each other's update history — it can look strong on the training verifier yet be *fragile* under any perturbation to either component.

Future work

- **Meta-RL over the action-selection policy.**
  Treat the harness-vs-weight selector as the object to be learned: each (trajectory, action, outcome) triple is a transition in an outer MDP. A self-improving system whose *improvement mechanism is itself self-improving*.

- **Finer-grained interleaving.**
  Let $\mathcal{F}$ trigger a weight update mid-harness-search, or resume harness exploration right after a gradient step — shrinking the lag between observing a plateau and acting on it.

Conclusion

- **SIA** is, to our knowledge, the only system that updates *both* the scaffold and the weights in a single self-improving loop.
- Task-agnostic: given a task spec + verifier, it produces an evolved scaffold *and* RL-adapted LoRA weights.
- Consistent gains over prior SOTA across several domains.

+25.1%	12.4% faster	+20.4%
LawBench	TriMul kernels	Denoising

Thank you